A Project Report

on

"Bi-Lingual Audio/Video Translator"

Submitted to
KIIT Deemed to be University

In Partial Fulfillment of the Requirement for the Award of

BACHELOR'S DEGREE IN
COMPUTER SCIENCE & SYSTEM ENGINEERING
BY

Rupkatha Chowdhury	2128044
Shaswata Dam	2128058
Tarunjit Saha	2128116
Sonit Kumar Swain	2128120

UNDER THE GUIDANCE OF
Dr. Ambika Prasad Mishra

KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
BHUBANESHWAR, ODISHA - 751024
March 2024

KIIT Deemed to be University

School of Computer Engineering
Bhubaneswar, ODISHA 751024

CERTIFICATE

"Bi-Lingual Audio/Video Translator"

submitted by

Rupkatha Chowdhury	2128044
Shaswata Dam	2128058
Tarunjit Saha	2128116
Sonit Kumar Swain	2128120

This certifies that the project is a record of their legitimate work completed toward partially fulfilling the requirements for the award of a Bachelor of Engineering degree (Computer Science & System Engineering) at KIIT Deemed to be University, Bhubaneswar. Under my direction, this work is completed in 2024.

Date: 10/04/2024

(Dr. Ambika Prasad Mishra)
Project Guide

Acknowledgements

We are extremely grateful to our tutor Dr. Ambika Prasad Mishra for his mentorship and continuous support in ensuring that this project meets its objectives since inception until its completion.

We would also like to acknowledge the contribution of all the participants who partook in this study and gave us data which was useful. The project could not have been accomplished without their cooperation and willingness.

Moreover, we would want to thank my colleagues as well as friends for being there with me right from the start of this work till it is over. Their input has been very vital through constructive criticisms leading to improvement of this report.

Lastly, we especially indebted to our family for their unyielding encouragement, care, and understanding. They have provided us with love and motivation throughout; hence we owe a lot of appreciation to them always being around us.

This project would not have been possible without your guidance and support.

Rupkatha Chowdhury
Shaswata Dam
Tarunjit Saha
Sonit Kumar Swain

ABSTRACT

In order to improve accessibility to digital content and get over language obstacles, this research introduces a bilingual audio/video transcription system with translation and speech synthesis interface. This technology offers real-time transcription, translation, and voice synthesis of audio and video content, making it available to a larger audience, since language is still a major barrier to accessing opportunities and vital services. The technology improves information availability in native languages, which is essential for professional services, healthcare, and education, by translating spoken content into bilingual text and audio. With its sophisticated speech recognition and text-to-speech conversion features, the system expands learning opportunities for non-native speakers and promotes diversity by translating content into a preferred language. This strategy is an essential first step toward a more fair digital environment, allowing people to participate

Keywords: Natural language processing (NLP), Machine learning, Real-time transcription, Translation, Speech synthesis, Transformers library.

Contents:

1. Introduction **1**

2. Basic Concepts/ Literature Review 2
2.1.Language Translation 2
2.2.Speech to Text 4
2.3.Text to Text 6
2.4.Text to Speech 8
2.5.Libraries Used 9
2.6.User Interface

3. Problem Statement & Requirement Specifications **11**
3.1. Problem Statement 11
3.2. Project Constraints 12

4. Implementation **14**
4.1. Code Snippets 14
4.2. Basic Architecture of our model 17

5. Result Analysis / Screenshots **21**

6. Conclusion and Future Scope **23**
6.1. Conclusion 23
6.2. Future Scope 24

7. References **26**

8. Individual Contributions **27**

9. Plagiarism Report **31**

List of Figures:

2.1.	Flowchart of whisper model	3
2.2.	Flowchart of M2M100 model	5
2.3.	Flowchart of the gtts model	7
4.1	Importing the required libraries	14
4.2	Importing Hugging face transformers library	14
4.3	Extracting audio from a video clip using MoviePy library	14
4.4	Code utilizing the whisper model for speech-to-text	14
4.5	Printing the transcripted text	15
4.6	Code for utilizing the M2M-100 library	15
4.7	Printing translated text	15
4.8	GTTS code for text - to - speech transition	16
4.9	Flowchart of the model architecture	17
4.10	Image representing the user interface	20

List of Tables:

5.1	Comparison Table	23

Chapter 1

Introduction

After all, the most central feature of human communication remains appropriate effective communication that can achieve purposes of information, knowledge sharing, and viewpoints of diverse groups. However, major impediments exist in communication at the level of languages as it cuts off direct, unobstructed access towards vital resources, services, and opportunities. An obvious application may be found within educational opportunities, professional services, and health-care management, to name a few. Information access in the native tongue is, thus, imperative.

This challenge increases with the onset of the digital and multi-media text, which commonly comes as audio and video formats. Much of these contents were not accessible and remain unreachable for nonnative speakers since there are translation and transcription tools available on the internet or through other modes of availability. For instance, educational materials or training programs and web content not in a person's first language can present unnecessary barriers to learning, growth, and even employment.

Promises of a bright way ahead are seen in the emergence of new AI and machine learning technologies. New audio and video content can take speech recognition, translation, and text-to-speech capabilities to translate into text format in two languages and give it an audio version for the first time. Ideas such as these are now no longer about crossing some language barriers but about letting people really be involved in digital content regardless of which language they speak.

The Bilingual Audio/Video Transcribing System with Translation and Speech Synthesis Interface addresses this urgent need by providing real-time transcription, translation, and speech synthesis of multimedia content. The bilingual capabilities of this system enhance accessibility, providing an easy and effective way for users to access content in their chosen language. The integration of translation and synthesis features also provides a smoother, more connected user experience adaptable to different language preferences and learning styles.

This project is one step closer to an equitable digital world, where language can no longer be a hindrance in discovering valued information and resources. The bilingual transcribing and translation system enables broader access and arms the user with the tools needed to enter content completely, either in a professional or personal learning environment.

Chapter 2

Basic Concepts/Literature Review

2.1 Language Translation: Language translation refers to the interpretation of written or spoken words from one language to another, maintaining its quality and context. Considering the current economy where almost every business is cross-cultural, translation is essential. Translation of these cultures is no longer solely in the hands of human beings; thanks to developments in technology, practically artificial intelligence has taken over translation business even that of natural languages. With the neural machine translation systems and other advanced linguistic models that tend to break the language barrier successfully, translations are now instantaneously produced.

Neural networks and deep learning architectures, like transformers, are used by contemporary language translation models, such as Google's M2M100 and OpenAI's Whisper, to capture linguistic nuances. These models can recognize patterns and connections both inside and between languages because they are trained on large multilingual datasets. These models preserve context, meaning, and intent by analyzing language at the sentence or paragraph level as opposed to word by word, producing translations that seem more authentic and human. Furthermore, multilingual models can manage translation directly between a variety of language pairs, obviating the need for error-prone intermediate translations.

There are several uses for automated language translation. Instant translation is revolutionizing cross-cultural interactions in corporate and educational settings, live video captioning, and customer service in real-time communication. In industries like healthcare or legal communication, where accuracy is crucial, translation systems are also indispensable. But there are still difficulties, especially when it comes to preserving idioms, slang, and cultural quirks that might differ greatly between languages. Future developments in machine translation technology will probably concentrate on enhancing the systems' cultural adaptation and accuracy for languages with fewer resources, making communication more inclusive and available to all.

2.2 Speech to Text: By transforming oral mode of communication to the written form, speech-to-text technology enables a range of voice-oriented applications like live transcription services, accessibility aids, etc. The Whisper model of Open AI, with its extensive multilingual approach, has changed the game in the field of speech to text in terms of increasing the accuracy

and versatility of the system's capabilities. With the ability to transcribe a huge number of languages, their dialects and accents, whisper is one of the most trustworthy models in the transcription field. Given its ability to learn from various datasets which encompass all forms of audio, people would say that Whisper is an effective transcription creation model. It can also identify voiced words under real-life conditions such as background sounds or noise, and varying quality of the audio.

The transformer-based architecture that Whisper employs is ideal for capturing the subtleties of natural speech. In comparison to earlier models, transformers handle input data in parallel, enabling the model to more effectively capture intricate linguistic patterns and dependencies within an audio sequence. Whisper is particularly accurate at transcribing languages with difficult grammatical structures or tone-based distinctions because of its architecture, which allows it to comprehend context over lengthy audio recordings. Furthermore, Whisper can translate and transcribe between languages directly, eschewing the conventional process of utilizing English as a mediator, thanks to its multilingual capabilities. For customers who require direct transcription across a variety of language pairs while maintaining the spoken content's natural flow and cultural context, this capability is essential.

Whisper's capabilities go beyond transcribing to include real-time applications such media live captioning, helping those with hearing loss, and promoting inclusive communication across linguistic divides. Additionally, it can help with workflows for content creation by automatically turning audio from lectures, podcasts, and meetings into text for simpler editing and distribution. Whisper is a breakthrough in speech-to-text technology because of its capacity to work in a variety of difficult contexts and languages. It also sets a new benchmark for the processing, storing, and application-wide use of voice data.

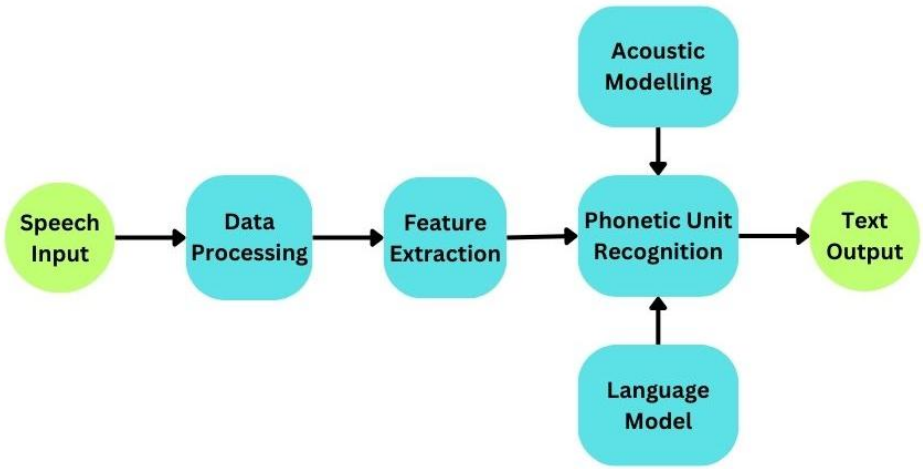

Figure 2.1: Flowchart of whisper model

The flowchart of the Whisper model outlines the systematic process by which it converts

spoken language into text. At the beginning, the audio input is processed and segmented, breaking down the audio into manageable frames that the model can interpret sequentially. Each audio segment then passes through Whisper's encoder, which uses a transformer-based neural network to extract features from the raw audio, identifying patterns such as phonemes, pitch, and cadence. These encoded audio features are then transferred to the decoder, another transformer network, which is trained to predict the most likely sequence of words corresponding to the given audio segment. In this phase, the decoder also incorporates language recognition and translation abilities if the model detects a non-English input, enabling direct translation if necessary. The model finally outputs the text transcription in the specified language, providing a real-time or post-processing transcription that is accurate and contextually aware. This structured flow ensures that the Whisper model can handle complex audio data with high precision, adapting to various languages and dialects through its multi-layered, attention-based framework.

2.3 Text to Text: Text-to-text language translation using M2M100 is a powerful approach to direct multilingual translation without relying on English as an intermediary language. M2M100, developed by Facebook AI, is one of the first fully multilingual translation models that supports translation between hundreds of languages. Unlike traditional models that often use English as a bridge language (which can result in translation inaccuracies), M2M100 directly translates from one language to another. This direct translation capability maintains context, cultural nuances, and linguistic specificity, especially for languages with distinct syntax or idioms, making it well-suited for preserving meaning across diverse language pairs.

The M2M100 model leverages a transformer-based architecture trained on a large multilingual dataset, covering 100 languages and over 2,200 translation directions. The model's architecture enables it to capture the linguistic and syntactic relationships within each language and between languages. By utilizing parallel data—where each segment of text in one language is aligned with its equivalent in another—M2M100 learns effective translation pathways between languages, allowing it to generalize across language pairs that may not have been explicitly seen in training. This design is especially impactful for low-resource languages, as it enables translations without the need for a vast amount of bilingual data, making high-quality, inclusive translations more accessible for lesser-represented languages.

Beyond simple translation, M2M100 has applications in real-time communication, multilingual content creation, and accessibility solutions, especially in areas with linguistically diverse people. M2M100 provides customers with a more accurate and culturally sensitive translation experience by avoiding the drawbacks of English-centric

translation systems. Because of its adaptability and flexibility, it can be used for anything from customer service to localizing instructional materials, enabling people and organizations to communicate successfully with a worldwide audience while minimizing meaning loss.

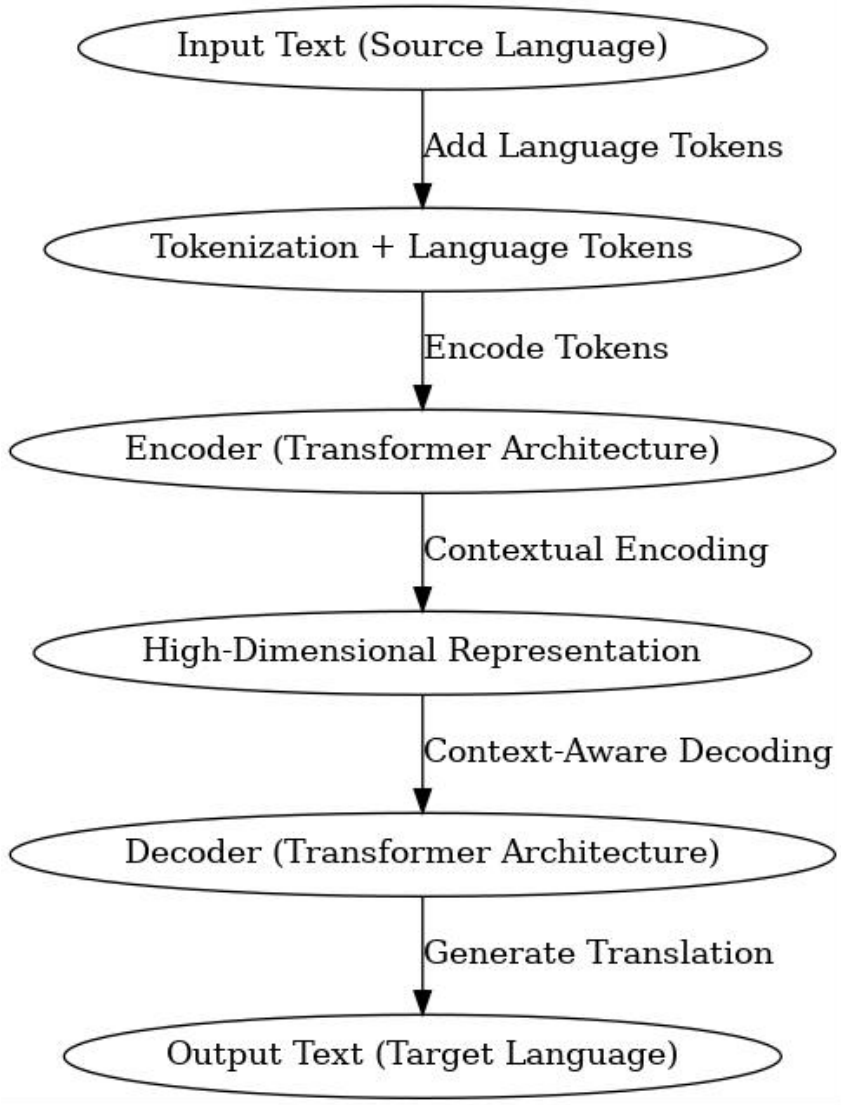

Figure 2.2: Flowchart of M2M100 model

In terms of flowchart, the process begins with the input text, which is preprocessed to segment it into manageable parts, including tokenization (splitting text into smaller units). The encoder of M2M100 then processes this tokenized text, transforming it into high-dimensional embeddings that represent the meaning and context of the words within each sentence. These embeddings are passed to a cross-attention layer where the model aligns words from the source language with their most likely equivalents in the target language, based on patterns it learned from multilingual data. The decoder then takes this aligned representation and generates text in the target language, effectively producing a fluent

and contextually accurate translation. The final output is the translated text, which can be fine-tuned or adjusted further if required. This structured flow allows M2M100 to perform direct translations efficiently, providing a versatile tool for multilingual communication across diverse languages.

2.4 Text to Speech: Advanced text to audio converters, otherwise referred to as text-to-speech technology (TTS), helps users who are listening inclined or need reading support engage with digital content more efficiently. One such well-known TTS application is Google Text-to-speech voice, 'gTTS' which helps changing the written text into very real sounding human voice in different languages and accents using Google services. Among many other occurrences, this tool can be used for audio guide design, conversion of teaching materials into accessible formats as well as provision of services to reading or visually impaired individuals. Also, with gTTS, the user does not have read but rather listen as gTTS enables the creation of a high-quality audio file that sounds more pleasing and entertaining than disturbing (in audio formats like MP3).

The ease of integration, multilingual compatibility, and variety of speech options of gTTS make it very beneficial. gTTS uses machine learning algorithms that improve pronunciation, intonation, and cadence to produce fluid, human-like speech by utilizing Google's sophisticated speech synthesis. Applications range from reading online sites aloud to turning written material into audio for language learners thanks to the model's multilingual capability. gTTS is a dependable option for people looking for authentic audio representation in their original languages because of its adaptability to accents and regional pronunciations.

In projects involving TTS, gTTS offers a straightforward and efficient approach to enhance user engagement and accessibility. For instance, integrating gTTS into educational tools enables students to listen to lessons in different languages, promoting a more immersive learning experience. It also serves as a valuable addition to mobile applications, enabling features like voice-driven user interfaces. Despite the straightforward functionality, gTTS can be limited in customization options compared to more advanced TTS systems. However, its simplicity, reliability, and high-quality output make it a go-to choice for developers aiming to add accessible voice features to their applications.

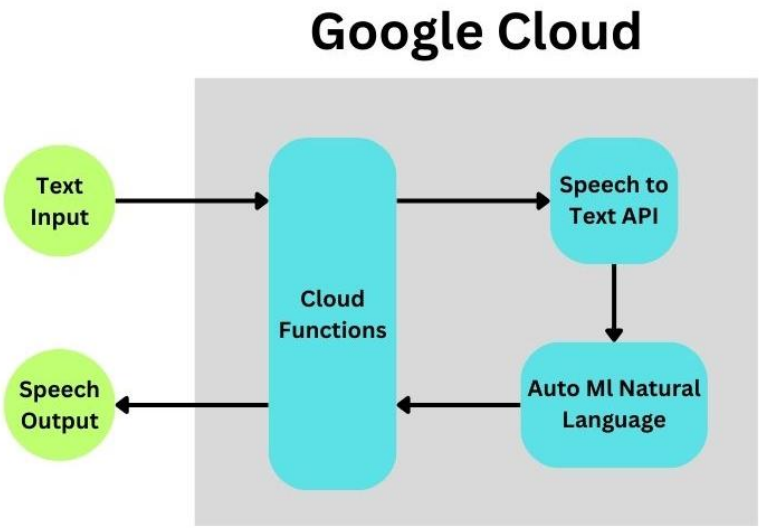

Figure 2.3: Flowchart of the gtts model

According to the gTTS (Google Text-to-Speech) flowchart, text speech synthesis is the process of turning written text into spoken language. The input text, which can literally be any statement or phrase that the user wants converted to sound, is where the flow starts. Here, the input undergoes language selection and accent customisation, giving the user the ability to select both languages and accents to enhance the audio output's quality and spoke relevancy. The text is then fed via Google's text-to-speech synthesis system, which uses sophisticated deep learning algorithms to construct a specific voice sound wave from the provided text while considering its grammatical, phonetic, and prosodic properties. Following processing, the speech can be played or used in programs in a variety of audio file formats, including MP3. Adding voice features to such projects is easy and straightforward because of gTTS's well-organized structure, which allows it to produce sounds that sound very much like human voices with minimal input.

2.5 Libraries used:

A. Transformers: The library has been created by Hugging Face, and is an active learning toolkit for all NLP related task. It offers access to ready to use transform models such as BERT, GPT, T5 and many that have been trained on large databases, hence permitting complexity of NLP operations without demanding high power machines. It can be said that transformers are very efficient for tasks such as text classification, translation, content generation, sentiment analysis and question answering. The library also offers tokenizers (which put text into chunks handled by the model) and models (which are already trained and can be used for purposes). The upside to this is that there is an API that allows users to load models and do whatever is required for the model with minimal effort in terms of coding. This library has powerful and persuasive currents thorough and thin, two models to these measures, respectively designed for machine translation and sentiment analysis.

B. IPython.display: The library IPython.display in Python, which is used in conjunction with the modules of IPython, is very powerful as it facilitates the inclusion of various media elements –such as sound or video, or pictures, or HTML files- within a Jupyter Notebook or other interactive Python session. The more specific sub-packages provide certain functions like display, Audio, Image, HTML, and Video, which allow users to add media and formatted text into their code cells. For example, Audio from the IPython display package may be used to embed sound files amongst other actions, this is most relevant to creation of multimedia, presentations and data examination. It also enhances the interactive and visualization aspects of the contents which is a great advantage for data scientists, scholars and computer programmers who wish to visualize or manipulate any media or outputs generated in their authoring environment and the results therein.

C. MoviePy: MoviePy is a powerful video editing library that makes it easier to handle audio and video files from Python. It allows several features including joining videos, cutting them, adding text and images, removing images or sounds, and other frame and sound extraction. Possession of this quality makes it very relevant in situations where one is engaging in the processing of audio-visual content like making videos, educational content, and even automating editing processes. For example, within MoviePy, it is simple for an end user's application to extract the audio track of a video, change it, and programmatically link video segments together. Lastly, it supports diverse architectures and applications thanks to inter multimedia library integration. Most of the video-related tasks that are tedious to developers as well as the end-users have been abstracted by MoviePy making it easy for developers to build applications that integrate

videos with other media effectively.

D. PyGame: Although it is also frequently used for interactive applications, animations, and basic simulations, PyGame is a cross-platform toolkit primarily created for game creation. It enables developers to create graphically interactive Python programs by offering modules that manage visuals, music, and user interactions. PyGame is appropriate for projects requiring real-time graphics or audio control since its sound module allows audio playback and its graphical modules manage image modification and screen rendering. Despite being designed for game design, PyGame is a popular option for instructional tools, audio-visual demos, and any project that needs an interactive user interface because of its ease of use and versatility. It is also a useful tool for adding multimedia elements, like interactive visualizations or audio playing, to Python programs because of its simple interface with other Python packages.

2.6 User Interface:

The "Multilingual Audio/Video Translator" project's user interface (UI) was developed with accessibility, usability, and functionality in mind, making it simple for users to translate audio and video material between languages. This one was constructed with Streamlit. Its well-defined framework guides users through the translation process methodically. Because of how dynamic and aesthetically pleasing the program becomes, even non-technical users can easily utilize Streamlit, a flexible Python toolkit that makes it easy to create web applications in a matter of minutes.

2.6.1 Key Features:

1. **File Type Selection:** Users can indicate the sort of media they wish to translate by choosing between audio and video file types. This option is crucial since it adjusts the backend processing according to the kind of file, which is particularly important for video files where audio extraction might be required.

2. **File Uploading:** The user interface has a drag-and-drop file upload section that supports several audio file formats, such as WAV, MP3, and M4A. Streamlit's st.file uploader feature makes it simple for users to submit files directly from their smartphone. User comprehension and usability are enhanced by clearly labeling the upload location with instructions.

3. **Translation Direction:** A dropdown menu allows users to choose the translation direction, such as "Hindi to English." This selection configures the translation model to work with the chosen source and target languages, making the UI adaptable to different language requirements.

2.6.2 Libraries Used:

Streamlit: Streamlit serves as the core library for developing the UI, providing pre-built components like buttons, file uploaders, and text input fields. It allows the application to be deployed as a web-based tool, giving users a real-time interface where they can interact with the model and view results instantly.

io.BytesIO and SoundFile (sf): io.BytesIO is used to handle byte streams, especially important for working with audio files in memory without writing to disk. SoundFile (imported as sf) is a library for reading and writing audio files in various formats, crucial for managing audio input and output in the application. Together, these libraries handle audio data efficiently, enabling smooth processing for translation tasks.

This UI effectively combines functionality with ease of use, guiding users through each stage of the translation process while leveraging Streamlit's dynamic components for an engaging experience. The overall interface design and choice of libraries ensure that the project remains accessible, scalable, and capable of delivering high-quality translations for various user needs.

Chapter 3

Problem Statement & Requirement Specifications

3.1 Problem Statement:

Multimedia content in today's digital world often suffers from language barriers that restrict access and understanding for different audiences. There is an increasing demand for real-time accessibility in various languages, but the existing solutions are inefficient and not consistent. Most platforms offer inconsistent translation quality, which cannot capture the subtle nuances of native languages and thus hampers understanding and engagement. There are also inefficient summarization methods that do not allow users to be able to use content efficiently.

Limited language support means non-English speakers cannot access it, so the reach of a global audience is decreased and information is not readily available. At the same time, user participation and interaction can be restricted due to inadequate real-time translation, which would act as a barrier for the non-native speaker to engage in live or recorded material. In addition, a lack of bias detection and a lack of cultural context in translation result in the potential for errors in content translation. Again, complexity in usage of existing translator tools is one of the significant factors that hinder greater acceptance of the translation technologies. For users that are deaf or visually challenged, inaccessible features exacerbate the problem and undermine the potential of creating of equal media access.

This project is meant to address the problems of creating a live-translated audio interface to translate content from English into native languages and vice versa, making engagement across linguistic and cultural boundaries more inclusive, accurate, and user-friendly.

3.2 Project Constraints:

1) Technical Limitations:

a) Model Performance: Depending on the intricacy of the original material and the languages being used, the model's accuracy in translating audio and video may vary.

b) Processing Speed: Effective processing skills are necessary to achieve real-time translation. Any delays could result in latency problems, which would be bad for the user experience.

2) Data Handling:

a) Data Privacy: Protecting user data privacy is essential, particularly when handling sensitive audio and video files. Adherence to regulations like GDPR is necessary.

b) Data Diversity: The model must be trained on a varied dataset to ensure accuracy across different accents, dialects, and speaking styles. A lack of diversity in the training data may result in biased or imprecise translations.

3) User Interface and Experience:

a) User Accessibility: Users especially those with disabilities, should be able to easily navigate and utilize the interface.

b) Bi-lingual Support: To enable users to communicate in their native language without compromising functionality, the user interface must support a variety of languages.

4) Dependency on External Resources:

a) Internet Connectivity: For real-time processing and API interactions, the model could require a steady internet connection, which could restrict its usability in places with erratic connections.

b) API Limitations: Using third-party APIs to provide translation services could have drawbacks in terms of speed, price, or service availability fluctuations.

5) Resource Constraints:

a) Computational Resources: For smaller businesses or projects in particular, the need for reliable hardware to provide real-time processing and translation may be a limiting constraint.

b) Project Timeline: The intricacy of integrating multiple components, verifying accuracy, and resolving unforeseen challenges can all have an impact on development timelines.

By addressing these limitations, the project can work toward creating a more effective bilingual audio-video translator that satisfies user requirements while maintaining accuracy and dependability.

Chapter 4

Implementation

4.1 Code Snippets:

```
[1] !pip install transformers
    from transformers import pipeline
    !pip install gTTS pygame
    !pip install git+https://github.com/openai/whisper.git -q -U
    from gtts import gTTS
    from IPython.display import Audio
    import pygame
```

Figure 4.1: Importing the required libraries

```
[2] from transformers import AutoTokenizer, AutoModelForSeq2SeqLM, pipeline
```

Figure 4.2: Importing Hugging face transformers library

```
[3] from moviepy.editor import VideoFileClip

    def extract_audio(video_file, audio_file):
        video_clip = VideoFileClip(video_file)
        audio_clip = video_clip.audio
        audio_clip.write_audiofile(audio_file)
        video_clip.close()

    video_file = "/content/Video.mp4"
    audio_file = "output_audio.wav"

    extract_audio(video_file, audio_file)
```

Figure 4.3: Extracting audio from a video clip using MoviePy library

```
[4] import whisper
    model = whisper.load_model("large")

    audio_path = "/content/output_audio.wav"

    result = model.transcribe(audio_path, language="en")
    transcription = result['text']
```

Figure 4.4: Code utilizing the whisper model for speech-to-text

```
print(transcription)

Today I will tell you about another most useful programming language called C++.
```

Figure 4.5: Printing the transcripted text

```
from transformers import M2M100ForConditionalGeneration, M2M100Tokenizer

model_name = "facebook/m2m100_418M"
translation_model = M2M100ForConditionalGeneration.from_pretrained(model_name)
tokenizer = M2M100Tokenizer.from_pretrained(model_name)

source_lang = "en"
target_lang = "hi"

def translate_text_m2m100(text, src_lang, tgt_lang):
    tokenizer.src_lang = src_lang

    encoded_text = tokenizer(text, return_tensors="pt")

    generated_tokens = translation_model.generate(
        **encoded_text, forced_bos_token_id=tokenizer.get_lang_id(tgt_lang)
    )
    translated_text = tokenizer.decode(generated_tokens[0], skip_special_tokens=True)
    return translated_text

english_text = transcription
hindi_translation = translate_text_m2m100(english_text, source_lang, target_lang)
```

Figure 4.6: Code for utilizing the M2M-100 library

```
[8] print("Translated text in Hindi:", hindi_translation)

Translated text in Hindi: आज में आपको एक और सबसे उपयोगी प्रोग्रामिंग भाषा के बारे में बताऊंगा जिसे सी ++ कहा जाता है. सी ++ एक शक्तिशाली, उच्च प्रदर्शन प्रोग्रामिंग भाष
```

Figure 4.7: Printing translated text

```python
def text_to_hindi_speech(text, output_filename="hindi_speech.mp3"):
    tts = gTTS(text=text, lang='hi')
    tts.save(output_filename)
    return output_filename

hindi_audio_file = text_to_hindi_speech(hindi_translation)
Audio(hindi_audio_file, autoplay=True)
```

0:05 / 0:55

Figure 4.8: GTTS code for text - to - speech transition

4.2 Basic Architecture of our model :

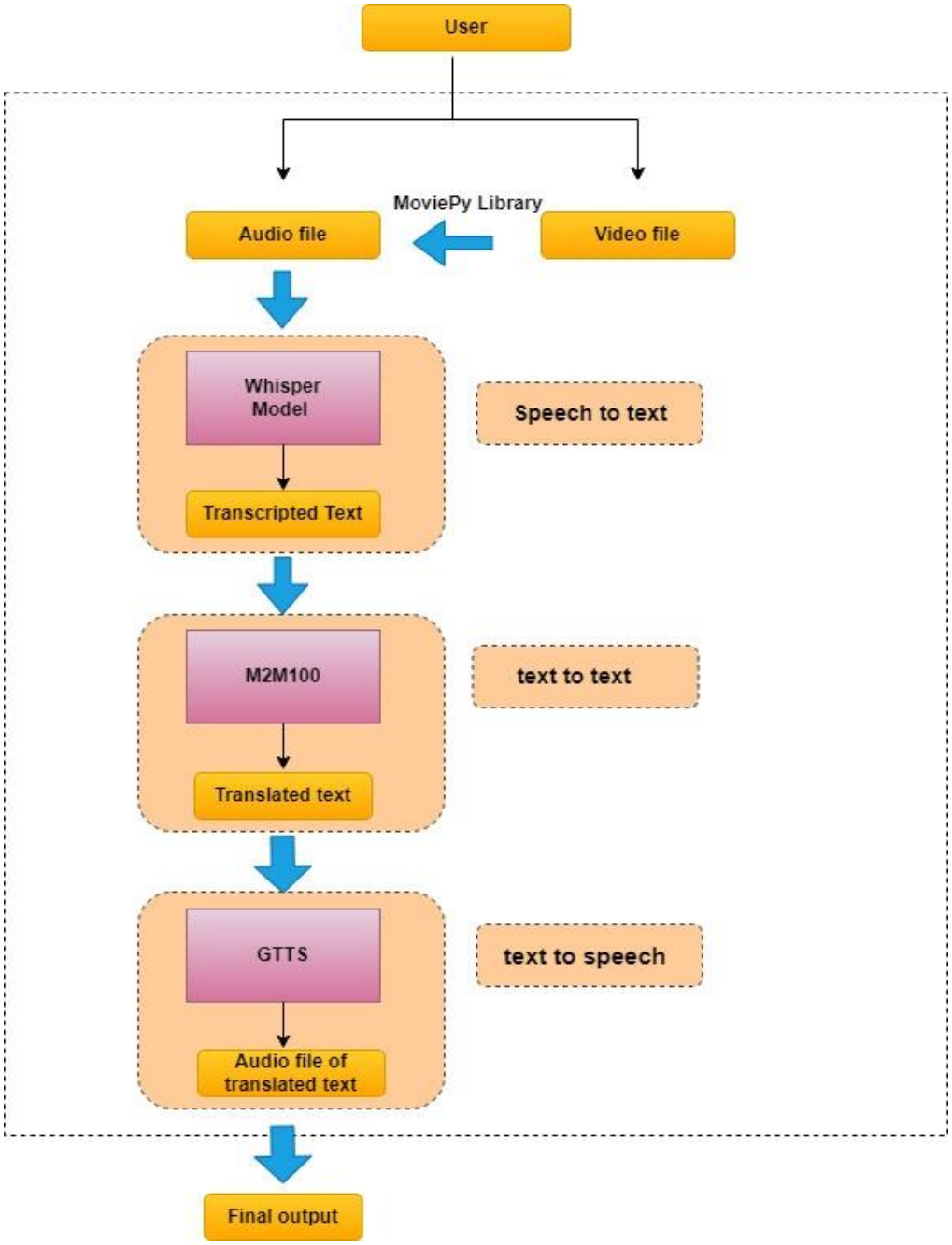

Figure 4.9: Flowchart of the model architecture

For an easy understanding of the work flow of our model, the entire framework can be broken down into five simple sections as follows:

1. Input:

The user can give input to our model by uploading an audio or a video file. In case the user uploads a video file, the audio from the uploaded video can be extracted by using the MoviePy library. This audio file now acts as the main input to our model.

2. Speech to Text - Using Whisper:

The whisper model was incorporated to obtain the transcripted text from the audio file.

The below points depict the reason for choosing whisper over other models:

I. With the help of active community assistance and an open-source framework for customisation, DeepSpeech uses deep learning algorithms to achieve respectable accuracy. In contrast to commercial solutions . But it offers limited language support and necessitates a large amount of labeled data and computational resources for training.

II. AssemblyAI is a model that offers a user-friendly API mkaking it simple to include into apps, supports a variety of accents, and offers high accuracy through deep learning models. However, because it is cloud-based, it requires an internet connection and has recurring expenses as a commercial product.

III. It offers speech to text transcription for ninety-nine languages hence making it a highly versatile model which can be used in advanced applications.

IV. It provides an accuracy of 97% for regional languages and 99% for other languages.

V. One of the greatest advantages of using the Whisper is that it can run efficiently on a CPU, hence this model can be deployed more broadly because it can be used on most devices, without requiring specialized hardware, such as GPUs. As a result, it could be used for real-time transcription and language processing tasks with low computational demands.

3. Text to Text - Using M2M100:

The output generated by the whisper model i.e. the text obtained from transcription acts as the input to the M2M 100 model. This model is used to translate the text from English-to-Hindi or vice versa.

The M2M100 model has following advantages which made it stand out amongst other models:

I. The NLLB200 model supports 200 languages .This model has the capability of excelling in low resource language .But it exceeded the free GPU resource offered by Google Collab hence it had to be replaced by a model which utilized lower resources.

II. One of the greatest advantage of mBART is that gives a high quality of translation among the language pairs . But , on the other hand relies on English as an intermediary language for translation. Hence, translations among less common languages might lose some meaning.

III. The M2M 50 serves as a smaller version of m2m100 which covers 50 languages including

Hindi . This models required very low computational resources . It provides an higher level of accuracy for translation between specific pair of languages . But this model has a reduced language range and hence it could not be integrated with the output of whisper ai.

IV. The M2M 100 model on the other hand provides translation for a wider range of regional languages.

V. It also provides a higher accuracy of around 85% for Hindi translations. Compared to other models it provides better translations and captures the nuances of the regional language well.

VI. M2M-100 also requires less computational resources and integrates well with output of whisper ai making it the best suited model to be integrated in our framework for the purpose of translation.

4. Translated text to Speech - Using GTTS:

The GTTS library offered by python was utilized at the final stage of our framework to generate the translated audio. The translated text obtained from the M2M100 model was used by GTTS to generate the audio which is the final output of our model.

The GTTS library was preferred over other models that performed Text to speech because of the following reasons:

I. Coqui TTS is a open sourced model that produces high quality speech from the desired text.It also supports a wide range of languages . Similarly another model called Mozilla TTS uses high-quality scientifically supported models that can generate expressive , lifelike speech . But both these models required fine-tuning using a custom-made dataset to perfrom TTS in native languages like Hindi.

II. eSpeak NG requires minimal computational resources to perform TTS . But it produced poor quality speech for regional languages hence this model was discarded.

III. Vakyansh is a model that is tailored to synthesize speech in Indic Language . But this could not be integrated in our framework because of a missing json file in their github repository.

IV. GTTS supports a variety of languages including native Indian languages making it best suited for our framework.

V. GTTS is also produces natural quality sounds which enhances the experiences offered by our application.

VI. It is a open-source library and hence the user does not have to pay for using it for basic TTS purposes.Hence the GTTS library was best suited for our framework .

5. User-interface using streamlit:

Streamlit and io library were used to develop an interactive user interface. This interface allowed the user to obtain a live translated audio from the video or audio file uploaded.

Figure 4.10: Image representing the user interface

The working of the user interface takes place as follows:

I. The user is given an option of uploading an image or an audio file. The audio file with extension of *.mp3* and *.m4a* can be passed while a video file with an extension of *.wav* can be uploaded. The file size has a limitation of 200 MB.

II. the user is given an option to translate from Hindi-to-English or English-to-Hindi.

III. our model is then accessed by making an API call.

IV. The model is then utilized by the interface to give the final output which is the translated audio file in English/Hindi

Chapter 5

Result Analysis:

The individual accuracy of the models are as follows:

A) Whisper which was used for speech-to-text transcription has an accuracy of 99%
B) M2M-100 which was incorporated for a english to hindi translations and vice versa has has an accuracy of 99%
C) GTTS used for text to speech purposes , produces an accuracy of 99%

Hence ,the accuracy of our model stands as the average of the above models that were incorporated to make the framework. Hence our model provides an accuracy of 99%

Our model was compared with other leading solutions that are available in the market. The comparison made it evident that our model is highly accurate, supports native video translation, and ensures natural sound quality in translation from English to Hindi. Other models require considerable setup and customization. Our model is ready to use with minimal configuration, which enables seamless integration and high performance, thus becoming an attractive choice for comprehensive, real-time multimedia translation.

The below tables depict the comparison between our model with other popular models that are present in the current market:

Translingo vs Microsoft Azure Translator and Speech Services vs IBM Watson Language Translator:

Feature	**Translingo**	**Microsoft Azure Translator and Speech Services**	**IBM Watson Language Translator**
1. Accuracy	Our model produces a cumulative accuracy of 99%	Provides an accuracy of 75-85 % for Hindi to english translation and 90 % for translation to english.	It gives an accuracy of 80-85% that can be improved with customization
2.Usability in multimedia content	Our model provides a user-friendly interaction as it supports both audio and video files	Effective for standalone audio or text but required additional setup for video content.	Does not work with video files
3.Quality of sound produced	Our model can generate high quality and natural sounding output	Native languages like Hindi require additional tuning for producing natural quality of sounds.	Require tuning for producing natural quality of sound
4. Native languages	Can handle regional dialects such as Hindi	Supports Hindi but quality of the sound is not natural	Struggles with Hindi expressions

Table 5.1: Comparison Table

Chapter 6

Conclusion & Future Scope

6.1 Conclusion:

The Bilingual Language Translation Transcribing System is the way to break the language barrier and equal opportunities. Using advanced technologies, including text-to-speech, bilingual language translation, voice detection, and compression, users can listen, watch, and participate in audio or video content with real-time translations regardless of the language used. This is crucial because bilingual content is a highly sought after commodity especially in the attempt to relate to others who belong in different demographics. The system addresses the need for there to be easy communication either in conversation, education or media use with no need to break a language-related barrier.

The system provides an interface that is usable such that it can allow free interaction through the various sorts of media. Most of its applications are used in schools, workplaces, as well as in social networks. Other coherent features are instant translation and summarization. Here, the content that the user is provided to focus entirely on may lack issues with language, whereby all this is possible, going to the benefits to users, due to cooperative interactions availing powerful communication tools for the economically disadvantaged and will enable access to such information that is not easily found. That is to say, this project extends the limit of learning by enhancing the engagement in communities.

Going forward, it can substantially contribute towards making huge changes as it can move ahead much further in educational opportunities for all and also personalize learning much better. It brings for businesses the most important advantage: communication without boundaries between teams across the nation. It encourages cooperation and effectiveness among business people. This project involves working with efforts towards a more just society in terms of the equal accessing of resources and information. Both accessibility and diversity serve this purpose. This bilingual language translation transcribing system stands at the forefront of a transformation movement which tries to break past a world that interprets understanding by means of language.

6.2 Future Scope:

Although the present version of the Bilingual audio/video Transcribing System is very promising, there are a few areas that can be improved and developed:

1.Expansion of Language and Dialect Support: A greater variety of languages and dialects should be included in future iterations of the system, with an emphasis on regional languages that are frequently disregarded in popular technologies. This growth will encourage inclusion and serve a larger audience.

2.Enhanced Accuracy and Reliability: To increase the accuracy of transcription and translation, the underlying models must be continuously improved by retraining with various datasets. Adaptive learning techniques and user feedback can also be used to improve performance over time.

3.User-Centric Customization: A more individualized experience that accommodates different needs and preferences can be produced by implementing features that let users change settings, such as language choices, summary length, and voice selections for text-to-speech.

4.Integration with Popular Platforms: Working together with already-existing platforms—like social media networks, online learning management systems, and video conferencing tools—will promote broader system acceptance and enable users to take advantage of the system's features without interruption in their favourite applications.

5.Real-Time Collaboration Features: By encouraging active participation and information sharing, collaborative solutions like shared notes or live discussions based on recorded content could improve interactivity in professional and educational settings.

6.Advanced Bias Detection and Mitigation: Ensuring the accuracy and impartiality of information, especially in delicate situations, will need the development of algorithms to identify and reduce biases in transcriptions and translations.

7.Accessibility Enhancements: The project's dedication to inclusivity will be strengthened by adding features created especially for people with impairments, like improved screen reader support and options for audio descriptions.

8.Performance Optimization and Scalability: Optimizing the system for increased speed, scalability, and performance will be essential as user demand increases. Larger information and

user bases can be accommodated by utilizing edge computing and investigating cloud computing solutions.

In conclusion, the Bilingual Audio/Video Transcribing System is well-positioned to have a great impact in the field of cross-language communication. As the world continues to increase global interactions, this project will play a pivotal role in breaking down language barriers and making information accessible to a diverse audience by addressing current limitations and pursuing ongoing enhancements.

References

1. https://huggingface.co/docs/transformers/en/model_doc/m2m_100dtyty
2. https://gtts.readthedocs.io/en/latest/
3. https://platform.openai.com/docs/guides/speech-to-text
4. https://open-speech-ekstep.github.io/
5. https://docs.coqui.ai/en/latest/models/xtts.html
6. https://discourse.mozilla.org/c/tts/285
7. https://github.com/espeak-ng/espeak-ng
8. https://huggingface.co/facebook/nllb-200-3.3B
9. https://huggingface.co/docs/transformers/en/model_doc/mbart
10. https://huggingface.co/facebook/mbart-large-50-many-to-many-mmt
11. https://deepspeech.readthedocs.io/en/r0.9/
12. https://www.assemblyai.com/

Bi-Lingual Audio/Video Translator

Rupkatha Chowdhury

2128044

1. Worked on text to speech recognition by implementing GTTS

2.Models that were researched upon are :
 a.Coqui TTS

 b.Mozilla TTS

 c. eSpeakNG

 d.Vakyansh

 e.GTTS

3.Contributed to Report writing by working on Chapter no 3 , 4 , 5 and making the flow chart of the working model .

Full Signature of Supervisor: Full signature of the student: Rupkatha Chowdhury

Bi-Lingual Audio/Video Translator

Shaswata Dam

2128058

1. Worked on models for text to text translation
2. Researched on the below listed models
   a. M2M 50
   b. mBART
   c. NLLB 200
3. Chose M2M 100 as the final model that was utilized in the framework
4. Integrated the models that were used in the framework i.e whisper , M2M 100 , GTTS
5. Standardization of code

Full Signature of Supervisor:

Full signature of the student: Shaswata Dam

Bi-Lingual Audio/Video Translator

Tarunjit Saha

2128116

1.Worked on models for speech to text transcription .

2.Reasearched upon the whisper model and utilized it to perform transcription

3. Other models that were looks into are :

a.Deep Speech (by Mozilla)

b.Assembly AI

4..Developed the user interface by utilizing streamlit and the io library

5..Integrated the models that were used in the framework i.e whisper , M2M 100 , GTTS

6..Developed an audio extractor from video file by utilizing MoviePy library .

7. Contributed to chapter 2 and 3 of report writing.

8.Standardization of code

Tarunjit Saha

Full Signature of Supervisor: Full signature of the student:

Bi-Lingual Audio/Video Translator

Sonit Kumar Swain

2128120

1.worked on text summarization using topics extracted from the translated text

2.identification of bias detection on a individual level by identifying which topics they are supporting and how

3.failure to identify the individual persons from the translated text overal decreasing the accuracy and preventing bias detection from functioning.Hence the text summarization part was not integrated in the main framework

4. Contributed to report writing by working on Chapter no 1,2,6.

Full Signature of Supervisor: Full signature of the student: